
Representing the SKI Combinator Calculus in the Nock Instruction Set Architecture

N. E. Davis `~lagrev-nocfep`
ZeroAttest, Groundwire Foundation

Abstract

It has been claimed that the Nock ISA closely corresponds to the `ski` combinator calculus dressed up for nouns. In particular, opcodes 2, 1, and 0 have been said to correspond to the combinators `S`, `K`, and `I`. This claim is usually stated as an analogy, and the details are elided. We here make it into an algorithm. We give a four-rule compiler from arbitrary `ski` terms to Nock formulas using only opcodes 0, 1, and 2, prove it correct under a fixed application convention, and exercise it on the usual worked examples: the identity laws, Church booleans, the `B`, `C`, and `W` combinators, and Church numerals performing real arithmetic. The compiled output is verified against an independent Nock 4K interpreter. The construction is a constructive proof that the Nock fragment of 0, 1, 2 opcodes is Turing-complete. We are explicit about the three properties it does *not* have—laziness, normal-form readback, and sharing—because each marks a boundary where the fragment is genuinely insufficient and additional opcodes are required.

Contents

Manuscript submitted for review.

Address author correspondence to `~lagrev-nocfep`.

1 Introduction

The Nock instruction set architecture is a combinator calculus sufficient to represent all computable functions. It has been a longstanding temptation to draw a direct analogy from the Nock ISA to the *ski* combinator calculus. This analogy claims that the Nock opcodes 2, 1, and 0 correspond to the combinators S, K, and I respectively. The analogy is usually stated informally, e.g. “[there are] some subtle differences to Nock’s expression of S as opcode 2 that we will elide as being fundamentally similar, but perhaps worthy of its own monograph” ([lagrev2025](#), [lagrev2025](#)). We here deliver upon that promise and convert an Urbit legend into a concrete demonstration.

The combinatory reading of Nock is well established. Smullyan-style combinator calculi build all computation from a handful of variable-free operators; the SKI system does it with three ([Curry1958](#), [Curry1958](#)), which is neither unique nor minimal but balances convenience with economy.

Put briefly, the S combinator (“substitute”) takes three arguments and applies the first to the third, then applies the second to the third, and finally applies the result of the first application to the result of the second. The K combinator (“k’onstant”) takes two arguments and returns the first, discarding the second. The I combinator (“identity”) takes one argument and returns it unchanged. The SKI system is Turing-complete, meaning that any computable function can be expressed as a combination of these three operators.

$$S \ x \ y \ z = x \ z \ (y \ z)$$

$$K \ x \ y = x$$

$$I \ x = x$$

The Nock ISA is also Turing-complete, and it is natural to ask whether the SKI combinators can be represented in Nock. The answer is yes, but the details are subtle. The analogy between Nock opcodes and SKI combinators is not a simple compilation procedure, but a homomorphism that preserves the structure of the computation.

A straightforward reading of Nock opcode 0 yielding I is uncontroversial, as [0 1] trivially returns identity. The reading

of opcode 1 yielding K is also straightforward, as $[1\ x]$ returns a constant function that discards its argument (subject). The reading of opcode 2 yielding S is more subtle, as it requires a fixed convention for how the compiled combinator receives its arguments. The “subtle differences” of **lagrev2025** elision matter more than they appear. The opcode-to-combinator analogy is not a compilation procedure. SKI terms are curried and higher-order: combinators are routinely partially applied, and a combinator may be passed as an argument to another combinator. A symbol-for-symbol substitution—writing 2 for S , 1 for K , and 0 for I , with appropriate transposition of arguments into subject and formula—produces nothing runnable the moment S has fewer than three arguments, which is almost always. What is needed is a *homomorphism*: a translation under which Nock application of compiled terms mirrors SKI reduction, with a fixed convention for how a compiled combinator receives its arguments.

We supply such a homomorphism as four rules with three small Nock formulas yielding the SKI combinators. The remainder of this article consists of worked examples, mechanical validation, a cost analysis, and an accounting of the limitations.

2 Preliminaries

We assume Nock 4K. Evaluation is the function $*[\text{subject}\ \text{formula}]$.

We require only three opcodes from the Nock ISA:

```
*[a 0 b]    ->  /[b a]           :: slot: fetch the noun
*[a 1 b]    ->  b                 :: quote: return b uncompiled
*[a 2 b c]  ->  *[*[a b] *[*[a c]]] :: eval: compute a subformula
```

plus the structural distribution (autocons) rule, which fires whenever a formula’s head is itself a cell:

```
*[a [b c] d] ->  [*[a b c] *[*[a d]]]
```

Slot address 1 denotes the whole subject, so $*[a\ 0\ 1] = a$.

A *value* is a Nock formula that consumes its argument as the *subject*. To apply a value v to an argument x , we evaluate

$\text{apply}(v, x) == *[x\ v]$

This operative convention inverts the lambda-calculus argument order, because in Nock the function is the formula and the argument is the subject.

3 Combinator Formulas

Under the convention of Section ??, each SKI combinator is a closed Nock formula. We give each, then verify its defining law by reduction.

3.1 I identity

$I\ x = x$ requires $*[x\ I] = x$, which is slot 1:

$$I = [0\ 1]$$

3.2 K constant

Applying ‘K’ to ‘x’ must yield a value that ignores its next argument and returns its value. Thus $K\ x\ y = x$ requires $*[x\ K] = [1\ x]$, which is built from subject x by autocons:

$$K = [[1\ 1]\ [0\ 1]]$$

For example,

$$*[x\ K] = [*[x\ [1\ 1]]\ *[x\ [0\ 1]]] = [1\ x]$$

Then for any y, $*[y\ [1\ x]] = x$, so $K\ x\ y = x$ holds.

3.3 S substitution

$S\ x\ y\ z = (x\ z)(y\ z)$. The combinator accumulates x, then y, then fires on z. We construct it bottom-up.

When the fully-applied $S\ x\ y$ finally receives z as subject, it must produce $(x\ z)(y\ z)$. Under the convention, $x\ z = *[z\ x]$ and $y\ z = *[z\ y]$, and the outer application is $*[*[z\ y]\ *[z\ x]]$. That is precisely opcode 2 with subject z, formula-b equal to y, formula-c equal to x:

$$S \ x \ y = [2 \ y \ x] \\ \therefore \ * [z \ [2 \ y \ x]] = * [* [z \ y] \ * [z \ x]] = (x \ z)(y \ z)$$

Working back one step, applying $S \ x$ to y (subject y , constant x) must build the noun $[2 \ y \ x]$:

$$S \ x = [[1 \ 2] \ [0 \ 1] \ [1 \ x]]$$

We check that $*[y \ S \ x] = [2 \ y \ x]$, since $*[y \ [1 \ 2]] = 2$, $*[y \ [0 \ 1]] = y$, and $*[y \ [1 \ x]] = x$. Working back the final step, applying S to x (subject x) must build $S \ x$; the first two parts are constants and the third, $[1 \ x]$, is built from x by the same autocons trick used for K :

$$S = [[1 \ [1 \ 2]] \ [1 \ [0 \ 1]] \ [[1 \ 1] \ [0 \ 1]]]$$

We check: $*[x \ S] = [[1 \ 2] \ [0 \ 1] \ [1 \ x]] = Sx$.

3.4 B composition

Any other combinator can be reached by elaborating its lambda definition to SKI and compiling, but the result is large. A named combinator is better derived directly, by the same method as the previous subsection: it becomes one builder layer per argument. The innermost layer is the opcode-2 application skeleton that fires on the final argument; each enclosing layer is an autocons that captures one argument and embeds it. We give B , C , and W .

For $B \ f \ g \ x = f \ (g \ x)$, the firing step computes f applied to $g \ x$, i.e. $*[x \ g] \ f$, which is opcode 2 with g in the formula-b slot and f quoted in the formula-c slot; capturing g then f gives:

$$Bfg = [2 \ g \ [1 \ f]] \\ \therefore \ * [x \ Bfg] = * [* [x \ g] \ f] = f \ (g \ x) \\ Bf = [[1 \ 2] \ [0 \ 1] \ [1 \ [1 \ f]]] \\ \therefore \ * [g \ Bf] = [2 \ g \ [1 \ f]] \\ B = [[1 \ [1 \ 2]] \ [1 \ [0 \ 1]] \ [[1 \ 1] \ [[1 \ 1] \ [0 \ 1]]]]$$

3.5 C transposition

The same procedure yields C (flip):

$$C = \begin{array}{c} [[1 \ 1 \ 2] \ [1 \ [1 \ 1] \ 0 \ 1] \ [1 \ 1] \ 0 \ 1] \\ :: \quad f \ x \ y \ -> \ f \ y \ x \end{array}$$

3.6 W duplicate

Likewise, W (duplicate):

$$W = \begin{array}{c} [[1 \ 2] \ [1 \ 0 \ 1] \ 0 \ 1] \\ :: \quad f \ x \quad \quad -> \ f \ x \ x \end{array}$$

3.7 Commentary

The one subtlety is how a captured argument is embedded, which is dictated by how the combinator uses that argument. If the argument is applied to a computed value (as B's f is applied to g x), it occupies a formula slot and is embedded quoted, [1 f]. If it is applied to the current subject (as W's f is applied to x) it must pass through as the live formula, embedded by slot, [0 1]. Quote where the argument is data; slot where the argument is the active function.

Notably, the value forms of B, C, and W use only Nock opcodes 0 and 1. The opcode 2 appears only in the formula they assemble at application time ([2 g [1 f]]), carried as quoted data. The combinators are pure construction; the application machinery is the structure they emit. Among the basis only S carries a 2 in its own body, because it performs two applications at once.

The size payoff over bracket abstraction is large (cell counts; see Section ?? for method):

	direct	via ski	ratio
B	11	208	19×
C	11	208	19×
W	6	130	22×

The two forms are extensionally identical on every input tested. For any combinator known by name, direct derivation is preferred and the abstraction tax is avoided entirely; bracket abstraction remains the general fallback for arbitrary lambda terms.

4 Compilation of the Homomorphism

Let $[[t]]$ denote a *closed* Nock formula whose product is the value of the SKI term t . The compiler is four rules:

$$\begin{aligned} [[S]] &= [1\ S] \\ [[K]] &= [1\ K] \\ [[I]] &= [1\ I] \\ [[A\ B]] &= [2\ [[B]]\ [[A]]] \end{aligned}$$

with S, K, I the formulas of §3. The combinators are *quoted* so that $[[\cdot]]$ always denotes a value-producing computation; application is opcode 2 applied to the two compiled operands.

Why quote, and why this order? Evaluating $[[[A\ B]]] = [2\ [[B]]\ [[A]]]$ against any subject s gives $**[s\ [[B]]]\ *[s\ [[A]]]$. Because $[[B]]$ and $[[A]]$ are closed, they ignore s and reduce to the values $\text{val}(B)$ and $\text{val}(A)$. The result is $**[\text{val}(B)\ \text{val}(A)]$ — apply value $\text{val}(A)$ to argument $\text{val}(B)$, i.e. A applied to B . The operands are reduced to values *before* application, which is what makes nested applications compose correctly; quoting a sub-application rather than evaluating it produces an off-by-one error (a residual K where the value was expected).

****Correctness.**** By induction on term structure. The base cases are §3. For application, the rule computes $\text{apply}(\text{val}(A), \text{val}(B))$, and by the induction hypothesis $\text{val}(A)$, $\text{val}(B)$ are the values of A , B ; by §2 this is the value of $A\ B$. The translation is therefore a homomorphism from SKI application to Nock evaluation, under the call-by-value strategy forced by opcode 2's eager semantics (see §8).

5 Conclusion

To summarize, why did you write it? Why do we care? What impact should it have on Urbit development?

Your bibliography is a separate BibTeX file. We use the plainnat bibliography style. You can use natbib citation commands like `\citep{wikipedia}` for parenthesized references.

Use `\citet{wikipedia}` for inline references. You can also use `\citeauthor{wikipedia}` for the principal author’s name.

“You can use traditional TeX—or LaTeX—representations”
(**Varney1987**, **Varney1987**).

“Or you can use fancy quotes—and symbols.”

End the article with the tombstone.☒